

ΠΛΗ311 Τεχνητή Νοημοσύνη - Εαρινό Εξάμηνο 2026 - Διδάσκων: Γεώργιος Χαλκιαδάκης

2^η Προγραμματιστική Εργασία-1^η Φάση (Σε ομάδες το πολύ 2 ατόμων)

Βάρος: 20% βαθμού μαθήματος

Παράδοση: έως 22/05/2026, ώρα 23:59

Οδηγίες: Ηλεκτρονική παράδοση συμπιεσμένου αρχείου μέσω του ιστοχώρου του μαθήματος (συμπεριλαμβανομένου και του κώδικα). Φροντίστε να είναι καλά οργανωμένο το παραδοτέο σας σε υποκαταλόγους με ευνόητα ονόματα. Τεκμηριώστε τον κώδικα σας επαρκώς. Συνοδέψτε την εργασία με μία το πολύ 12-σέλιδη αναφορά όπου εξηγείτε τις επιλογές σας και περιγράφετε τα αποτελέσματά σας. Θα βαθμολογηθείτε και για την ποιότητα της αναφοράς σας - φροντίστε λοιπόν να είναι κατανοητή, να απαντάει τα ερωτήματα της εκφώνησης, και να αντιστοιχεί σε / περιγράφει ορθά τον κώδικά σας.

Μπορείτε ελεύθερα να χρησιμοποιείτε έτοιμο κώδικα για επιμέρους κομμάτια των προγραμμάτων σας αρκεί να αναφέρετε επακριβώς τις πηγές σας. Απαγορεύεται ρητά η χρήση LLM για παραγωγή κώδικα.

Κατά τα λοιπά, αντιγραφή συνεπάγεται άμεσο μηδενισμό στο μάθημα.

Εισαγωγή

Στην παρούσα εργασία, καλείστε να αντιμετωπίσετε το πρόβλημα της τοπικής αναζήτησης, προκειμένου να εντοπισθεί η καλύτερη διαδρομή που ενώνει δύο καταστάσεις. Σε αντίθεση με την προηγούμενη εργασία, εδώ, δεν θα υποτεθεί γνώση των “δυναμικών” εξισώσεων που διέπουν την μετάβαση μεταξύ καταστάσεων (transition dynamics) του περιβάλλοντος. Τα περιβάλλοντα στα οποία θα κληθείτε να υλοποιήσετε τους ζητούμενους αλγορίθμους, μεταβάλλονται με έναν μη-στατικό (non-stationary) τρόπο, συνεπώς και η εκμάθησή τους θα αποτελούσε ένα δύσκολο πρόβλημα.

Πιο συγκεκριμένα, καλείστε να υλοποιήσετε τους αλγορίθμους:

- Hill Climbing
- Random Restart Hill Climbing
- Simulated Annealing

Οι παραπάνω αλγόριθμοι, θα εφαρμοστούν στα προβλήματα [CartPole](#) και [MountainCar](#). Η προσομοίωση των περιβαλλόντων θα γίνει μέσω της βιβλιοθήκης [ns-gym](#), η οποία στηρίζεται στο gymnasium. Για την χρήση αυτής της βιβλιοθήκης προτείνεται η απευθείας χρήση python (και συστήνεται η χρήση των εκδόσεων 3.10 ή 3.11). Επιπλέον, το reward function (συνάρτηση απόδοσης αμοιβής) των συγκεκριμένων περιβαλλόντων έχει μεταβληθεί, με τη χρήση των wrappers που βρίσκονται στον αντίστοιχο φάκελο.

Στον κώδικα που σας δίδεται, υπάρχουν δύο βασικά folders (test & agents), μέσα στα οποία περιέχονται τα αρχεία που καλείστε να αλλάξετε. Πιο συγκεκριμένα, στο αρχείο `agents.local_search_agent` έχει υλοποιηθεί μια “abstract” κλάση που καλύπτει τις βασικές λειτουργίες του Local Search πράκτορα, όπως αυτή του `calculate_value` και `project_act`. Τέλος, έχει υλοποιηθεί η μέθοδος `find_best_route()`, στην οποία ο local search agent τροφοδοτεί την επιλεγμένη κίνησή του, στο περιβάλλον. Επιπλέον, μέσω της μεθόδου κρατούνται λίστες με τις καταστάσεις που έχουν επισκεφθεί, τις αντίστοιχες επιβραβεύσεις κτλ. Αυτή η μέθοδος, είναι υπεύθυνη και για την οπτικοποίηση του περιβάλλοντος. Για την ορθή χρήση αυτής της μεθόδου από κάθε πράκτορα, χρειάζεται η υλοποίηση της μεθόδου `act` (η μέθοδος που αποφασίζει την επόμενη κίνηση), όπως αυτή προτείνεται από κάθε πράκτορα.

Ζητούμενα

Στην παρούσα άσκηση, σας ζητάτε η δημιουργία των τριών προαναφερθέντων πρακτόρων, οι οποίοι θα πρέπει να κληρονομούν την λειτουργικότητα του `LocalSearchAgent` και υλοποιούν την μέθοδο `act()`. Για τον Simulated Annealing πράκτορα, μπορείτε να πειραματιστείτε με διαφορετικές στρατηγικές στον τρόπο που η παράμετρος θερμοκρασίας μεταβάλλεται. Η κλήση της `LocalSearchAgent.find_best_route()` πρέπει να γίνει σε ένα **νέο** αρχείο το οποίο θα δημιουργήσετε στο φάκελο `test`.

Πειραματική διαδικασία

Για τον έλεγχο των υλοποιήσεων σας, σας ζητάτε να τρέξετε την παρακάτω πειραματική διαδικασία για διαφορετικό αριθμό μέγιστου αριθμού βημάτων (`max_timesteps = 10, 100, 500, 1000`) για κάθε επεισόδιο.¹ Τρέξτε 100 επεισόδια για `max_timesteps = i` και για κάθε υλοποιημένο αλγόριθμο. Σε κάθε επεισόδιο αθροίστε τις επιβραβεύσεις όπως αυτές έχουν επιστραφεί από το περιβάλλον, καταλήγοντας έτσι με 100 αθροιστικά rewards (100 cumulative episodic rewards). Τέλος, υπολογίστε τον μέσο όρο από τα 100 αθροιστικά rewards. Επαναλάβετε αυτή την διαδικασία για όλα τα `max_timesteps` και όλους τους αλγορίθμους. Δημιουργήστε ένα γράφημα, που θα απεικονίζονται τα αποτελέσματα σας.

Επιπλέον, για `max_timesteps = 100`, και 1000 επεισόδια, δημιουργήστε ένα γράφημα που θα απεικονίζει τις αθροιστικές επιβραβεύσεις για κάθε επεισόδιο, προκειμένου να αξιολογηθεί η συμπεριφορά του κάθε πράκτορα, όσο ο χρόνος κυλάει και συνεπώς το περιβάλλον μεταβάλλεται. Αναμένουμε δύο εκδοχές των παραπάνω γραφημάτων, μία για κάθε περιβάλλον.

Παραδοτέα αρχεία (σε ένα συμπιεσμένο αρχείο μορφής .zip)

- Τελικός κώδικας της υλοποίησής σας (έως 10% της τελικής βαθμολογίας του μαθήματος)

¹ Μπορεί να αλλάχθει κατά την αρχική δημιουργία του `gymnasium environment` με τη χρήση του argument “`max_episode_timesteps`”

- Αναφορά (έως 10% της τελικής βαθμολογίας του μαθήματος)

Οδηγίες εγκατάστασης

Κατεβάσετε τον κώδικα από το github, με χρήση της εντολής:

```
git clone https://github.com/leoBakop/Al-agents-in-ns-gym
```

Έπειτα, ακολουθήστε της οδηγίες εγκατάστασης, όπως αναγράφονται στο README

Χρήσιμη βιβλιογραφία (συμπληρωματική των διαλέξεων)

[Russel's and Norving's Book, Κεφάλαιο 4]:

<https://people.engr.tamu.edu/guni/csce625/slides/Al.pdf>

[Ns gymnasium]: https://github.com/scope-lab-vu/ns_gym